

closure_evidence_20260925

BOB-226 — closure evidence (2026-09-25)

Item

scripts/ownership_repair.sh's repair walk fixes ownership on files/directories whose uid is not the operator. The existing unit suite (tests/unit/test_ownership_repair.sh) seeds a foreign uid only onto FILES and SYMLINKS — never onto INTERIOR DIRECTORIES (the one case that does seed a foreign-owned root, Case 22, is on the injected-failure path, not the repair-walk success path). Production's real first-start repair exercises exactly the untested shape: interior directories with a foreign uid.

Acceptance criterion: an integration-layer test that seeds foreign-owned INTERIOR directories via podman unshare (or the mechanism tests/ownership/test_container_writes_owned_files.py already establishes), runs the actual repair (scripts/ownership_repair.sh), and asserts the POST-repair state: ownership corrected on those interior directories, AND mode preserved.

1. Confirmation the podman unshare mechanism actually works here

```
$ which podman docker
/usr/bin/podman
$ id -u; id -g
1000
1000
$ podman info --format '{{.Host.Security.Rootless}}'
true
$ grep "^$(whoami):" /etc/subuid /etc/subgid
```

```
/etc/subgid:milosvasic:100000:65536
/etc/subuid:milosvasic:100000:65536
```

Live probe (before writing the test), on a real directory tree with content created BEFORE the chown, matching the fixture shape the test uses:

BEFORE:

```
.../bob226_probe uid=1000 gid=1000 mode=775
.../bob226_probe/outer uid=1000 gid=1000 mode=775
.../bob226_probe/outer/inner uid=1000 gid=1000 mode=775
.../bob226_probe/outer/inner/leaf.txt uid=1000 gid=1000 mode=664
---running podman unshare chown on interior dir 'outer' (non-recursive, dir itself only)---
```

AFTER (host view):

```
.../bob226_probe uid=1000 gid=1000 mode=775
.../bob226_probe/outer uid=100000 gid=100000 mode=775
.../bob226_probe/outer/inner uid=1000 gid=1000 mode=775
.../bob226_probe/outer/inner/leaf.txt uid=1000 gid=1000 mode=664
---can operator write into outer now?---
touch: cannot touch '.../bob226_probe/outer/should_fail':
Permission denied
write denied (expected, matches production defect shape)
---cleanup via unshare---
done
```

This confirms: (a) podman unshare chown 1:1 -- <dir> produces a REAL, not-operator-owned INTERIOR directory (uid 100000, the first subordinate uid — exactly the production defect's identity class), (b) content created inside it *before* the chown survives untouched (so the fixture never needs write access to a directory it no longer owns — the exact constraint the BOB-226 item names for why cases 8/9/18 in the unit suite could not be extended this way), and (c) the operator genuinely loses write access afterward, reproducing the real defect shape rather than a proxy for it.

Cleanup mechanism (podman unshare rm -rf <root>) confirmed to remove the WHOLE tree, including the outer, operator-owned root, in one call:

```
$ podman unshare rm -rf "$WORK"
rc=0
REMOVED ENTIRELY: $WORK
```

2. Files created / modified

Path	Change
tests/ownership/ test_repair_interior_directories.py	NEW. Integration-layer pytest test.
docs/qa/B0B-226/ closure_evidence_20260925.md	NEW. This file.

scripts/ownership_repair.sh was **not modified** — see §4.

3. RED-then-GREEN

3a. GREEN — the shipped, unmodified script

```
$ .venv/bin/python -m pytest tests/ownership/  
test_repair_interior_directories.py -v \  
    --import-mode=importlib -p no:cacheprovider  
===== test session starts  
=====  
platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0  
collecting ... collected 1 item  
  
tests/ownership/  
test_repair_interior_directories.py::test_repair_reaches_interior_  
PASSED [100%]  
  
===== 1 passed in 0.73s  
=====
```

Re-run to confirm determinism (§11.4.50): also 1 passed, no flake observed across two independent runs (this section’s run + the run captured during authoring, both green).

3b. RED — proving the oracle actually detects the untested shape

Per §11.4.115(F)/(G), a RED must be traceable to the real defect shape, not a synthetic one. Rather than modifying the shipped scripts/ownership_repair.sh (out of scope unless a genuine bug is found — see §4), the RED was produced against a **byte-for-byte copy** of the real script with ONE line mutated: the walk’s item-selection predicate

gained ! -type d, which excludes every directory from repair while leaving file/symlink repair untouched — i.e. it mechanically disables exactly the capability this item is about, and nothing else.

```
$ grep -n 'find_args+=(! -uid' scripts/ownership_repair.sh
1099:     find_args+=(! -uid "${OP_UID}" -printf
    '%U\t%G\t%m\t%p\0')
```

```
$ diff -u scripts/ownership_repair.sh /tmp/.../bob226_red/scripts/
ownership_repair.sh
--- scripts/ownership_repair.sh
+++ /tmp/.../bob226_red/scripts/ownership_repair.sh
@@ -1096,7 +1096,7 @@
     # owns is left alone, so a deliberate shared-group or setgid
    arrangement is
     # no longer silently dismantled by a repair that was never
    asked to.
     #
-----
-    find_args+=(! -uid "${OP_UID}" -printf '%U\t%G\t%m\t%p\0')
+    find_args+=(! -type d ! -uid "${OP_UID}" -printf
    '%U\t%G\t%m\t%p\0')
```

Test run against that patched copy (via SCRIPT pointed at the mutated file — the test's own fixture-building, seeding, scope file and invocation are otherwise byte-identical to the GREEN run):

```
$ .venv/bin/python -m pytest /tmp/.../bob226_red/
test_repair_interior_directories_RED.py -v \
    --import-mode=importlib -p no:cacheprovider
...
>         assert wrong_dir_uid_after == operator_uid, (
            f"interior directory {wrong_dir} is still owned by
uid "
            f"{wrong_dir_uid_after} after the repair reported
success – the "
            f"repair-side walk does not reach foreign-owned
INTERIOR "
            f"DIRECTORIES, only the files/symlinks the unit
suite covers "
            f"(BOB-226)"
        )
E         AssertionError: interior directory /tmp/ownership-
repair-interior-9x6gg0np/declared_scope/wrong_dir is still owned
by uid 100000 after the repair reported success – the repair-side
walk does not reach foreign-owned INTERIOR DIRECTORIES, only the
files/symlinks the unit suite covers (BOB-226)
E         assert 100000 == 1000

FAILED .../
test_repair_interior_directories_RED.py::test_repair_reaches_inter:
```

```
===== 1 failed, 1 warning in 0.70s
=====
```

The RED is for the **right, narrow reason**: the mutated repair still exits 0 (“reported success”) because directories are simply never discovered as needing repair — it is not a crash, not a script-bug fail (§11.4.1), and it fails on the FIRST directory-uid assertion, exactly the load-bearing assertion this item exists to add. The seeding, control-needle, and pre-repair `PermissionError` assertions all still pass identically in both runs, proving the failure is isolated to the repair’s directory-handling and nothing else in the fixture.

Restoring the unmutated script (§3a) turns the identical test GREEN again.

4. Was there a genuine bug in `ownership_repair.sh`?

No — the script was already correct; only the test coverage was missing.

The walk’s item-selection predicate (`scripts/ownership_repair.sh:1099, find_args+=(! -uid "${OP_UID}" -printf ...)`) carries **no -type filter at all**, so by the script’s own source it already selects files, directories, AND symlinks uniformly — nothing in the shipped code special-cases directories out of repair. Running the real, unmodified script against the new fixture (§3a) confirms this empirically: both the top-level and the nested interior directory are repaired to the operator’s uid, their mode is byte-identical before and after, and the files inside them are repaired too, all through the real invocation path (`bash scripts/ownership_repair.sh --scope ... --state-dir ...`), with the `unshare_chown_paths()` fallback exercised (a plain `chown` on a `uid-100000` directory fails `EPERM` as an unprivileged operator; the script’s own fallback then re-enters the namespace and succeeds — this is the “production repair path” the unit suite’s own header identifies as previously exercised only by files, never by directories).

One genuine, unrelated Unix-permissions finding surfaced while building the fixture and was corrected in the TEST (not the script): an interior directory’s mode denying “other” **read** (e.g. `0o751`) makes `find` itself report `Permission denied` when trying to enumerate that directory’s contents — `find` runs as the unprivileged operator for *discovery*; only the individual `chown` falls back to the namespace on failure. This is a property of how Unix directory permissions interact

with an unprivileged `find`, not a defect in `ownership_repair.sh`'s logic, and it does not match the realistic production shape (containers create directories under an ordinary umask, typically `0755/0775`, which keeps “other” readable). The fixture now uses `0o745 / 0o715` (both retain `r-x` for “other” so `find` can traverse, both distinctive so a silent mode-reset would still be caught, and both still deny write so the pre-repair `PermissionError` assertion continues to reproduce the real defect).

5. Full existing unit suite — confirms no regression

Although `scripts/ownership_repair.sh` was not touched, the full existing suite was run anyway for confidence, since it shares `scripts/lib/ownership.sh` with the new test's invocation path:

```
$ bash tests/unit/test_ownership_repair.sh
...
RESULT: 153 passed, 0 failed, 0 skipped
```

6. Assumptions the item text did not fully specify

- **Which directory (§3b's mutation target).** The item did not name the exact code path that would need to change to reach interior directories; the script's own source (no `-type` filter) made clear none was needed, so the RED demonstration mutation targets the one line that *would* need to exist if the walk were narrower than it actually is.
- **Directory permission mode for the fixture.** The item asked for “mode preservation” without specifying a mode. The realistic production shape (container-created directories under an ordinary umask, readable by “other”) was chosen over an arbitrary restrictive mode, both because it matches production and because an overly restrictive mode (first attempt: `0o751`) broke `find`'s own discovery step for reasons unrelated to the repair under test — see §4.
- **Placement: `tests/ownership/` over `tests/integration/`.** `tests/ownership/__init__.py` explicitly documents that package's tests are kept out of `tests/integration/` because that package's `conftest.py` installs an autouse fixture requiring the full live stack to be up, which this test does not need (it only needs a container *runtime*, not the compose stack). The existing sibling test

(`test_container_writes_owned_files.py`, which the item cross-references) already lives there for the identical reason, so the new file follows it.

- **Not adding an out-of-declared-scope negative control.** The bash suite's Case 3 already covers the scope-fence property; this test focuses narrowly on the acceptance criterion (interior-directory ownership + mode preservation) rather than re-proving already-covered ground, per §11.4.83 (evidence, not gold-plating).